

PULSO

Community Social App

Project Requirements Document

Mobile Applications Development — Semester 2 Group Project

Project Duration: 2 Weeks (14 Days)

Team Size: 3 – 4 Students

Total Points: 100 pts

Backend Platform: Supabase (Auth, Database, Storage, Realtime)

State Management: Riverpod (required)

Testing: Unit + Widget tests (required)

Assessment: Rubric-Based Document + Artifact Submission

1. Project Overview

Pulso is a mobile community social app built with Flutter and powered by Supabase as its cloud backend. The name "Pulso" is a Filipino word for "pulse" — reflecting a living, breathing feed of community moments.

Students will design, build, test, and document a working social application where users can register, share posts with images, react to content, leave comments, and follow other community members — all synchronized in real time through Supabase.

Why this project? Every major social product in production today relies on cloud authentication, cloud storage, real-time data synchronization, and access control policies. Building Pulso puts all four of those foundations in your hands at once — with a tech stack actively used by startups and product studios in the Philippines and internationally.

2. Learning Objectives

By the end of this project, each team member must be able to:

1. Integrate Supabase authentication into a Flutter app with persistent session management
2. Design a relational Supabase database schema with foreign key relationships and proper constraints
3. Implement Supabase Realtime to push live updates to the UI without manual refresh
4. Upload and retrieve files (images) using Supabase Storage with signed/public URLs
5. Write and apply Row Level Security (RLS) policies to protect user data at the database level
6. Manage application state using Riverpod providers that react to Supabase data changes
7. Write unit tests for service and repository layers and widget tests for key UI components

3. Technical Stack

Component	Technology / Constraint
Framework	Flutter (latest stable)
Language	Dart
Backend	Supabase (cloud-hosted, free tier)
State Management	Riverpod (flutter_riverpod) — REQUIRED
Navigation	GoRouter or Flutter's Navigator 2.0 — team's choice
Auth	Supabase Auth (email/password)
Database	Supabase PostgreSQL + Realtime
Storage	Supabase Storage (images bucket)
Testing	flutter_test, mocktail or mockito
Version Control	Git + GitHub (public repository)
Key Packages	supabase_flutter, flutter_riverpod, riverpod_annotation, image_picker, cached_network_image

Riverpod is non-negotiable. All Supabase-driven state — auth state, feed data, comments, likes, follow state — must flow through Riverpod providers. Raw `setState()` calls are only acceptable for purely local ephemeral UI state (e.g., toggling a text field's visibility). Any submission that manages Supabase data outside Riverpod will be penalized under the Riverpod State Management rubric criterion.

4. Feature Specifications

All six features listed below are required. There are no optional features. A feature is considered complete only when it is connected to Supabase — local/in-memory stubs will not receive credit.

4.1 User Registration & Login

- Email and password registration via Supabase Auth
- Email and password login with session persistence across app restarts
- Logout that fully clears session and navigates back to login
- User-friendly error messages for invalid credentials, duplicate email, and network errors
- A profiles record is automatically created on first login/signup (trigger or manual insert)

4.2 User Profile with Avatar Upload

- Editable profile screen showing username, bio, and avatar
- Avatar image picked from device gallery and uploaded to Supabase Storage
- Avatar URL stored in the profiles table and displayed via `cached_network_image`
- Profile visible to other users (public read via RLS)

4.3 Post Feed with Image Upload

- Authenticated user can create a new post with a caption and an image
- Image uploaded to Supabase Storage; URL stored in the posts table
- Feed screen displays all posts in reverse-chronological order
- Each post card shows: user avatar, username, post image, caption, like count, comment count
- Feed is paginated or uses infinite scroll (minimum 10 posts per load)

4.4 Likes / Reactions

- Authenticated users can like or unlike any post
- Like count updates in real time via Supabase Realtime (no manual refresh required)
- A user cannot like the same post twice (enforced at DB level with a UNIQUE constraint)
- The like button reflects the current user's like state (filled vs unfilled icon)

4.5 Comments

- Authenticated users can add a text comment to any post

- Comment list updates in real time when a new comment is inserted
- Each comment shows: commenter avatar, username, comment text, timestamp
- The post owner can delete any comment on their post; comment author can delete their own

4.6 Follow / Unfollow

- Authenticated users can follow and unfollow other users
- A user's profile shows their follower count and following count
- A follows table tracks the follower_id and following_id relationship
- The follow button on a profile reflects current follow state (Follow vs Following)

5. Supabase Architecture Requirements

5.1 Database Schema

Students must design and implement the following tables in their Supabase project. Additional tables or columns are allowed but must not replace any of the required ones.

Table	Key Columns	RLS Required
profiles	id (FK auth.users), username, bio, avatar_url, created_at	SELECT: public; UPDATE: owner only
posts	id, user_id (FK), image_url, caption, created_at	SELECT: public; INSERT/UPDATE/DELETE: owner only
likes	id, post_id (FK), user_id (FK), created_at	SELECT: public; INSERT/DELETE: authenticated + owner only
comments	id, post_id (FK), user_id (FK), body, created_at	SELECT: public; INSERT: authenticated; UPDATE/DELETE: owner only
follows	id, follower_id (FK), following_id (FK), created_at	SELECT: public; INSERT/DELETE: follower only

Schema Export Required: Students must include a screenshot or SQL dump of their final schema in their submission. Supabase Dashboard > Table Editor > Export, or use the SQL Editor to run \d and capture output.

5.2 Supabase Auth

- Use supabase_flutter's built-in auth methods: `signUp()`, `signInWithPassword()`, `signOut()`
- Listen to `onAuthStateChange` stream via a Riverpod `StreamProvider` to drive auth-gated navigation
- Store the current user's id and link it as the `user_id` foreign key in all user-owned tables

5.3 Supabase Realtime

- Subscribe to the `posts_likes` channel to receive live INSERT/DELETE events on the likes table
- Subscribe to the `post_comments` channel to receive live INSERT events on the comments table
- Unsubscribe from channels when the widget/provider is disposed (prevent memory leaks)
- Realtime must be enabled in the Supabase Dashboard under Database > Replication for the required tables

5.4 Storage Buckets

- Create an avatars bucket for user profile photos
- Create a posts bucket for post images
- Both buckets should be configured as public or use signed URLs — document your decision
- File naming convention: `{userId}/{uuid}.jpg` to prevent collisions

5.5 Row Level Security (RLS) Policies

RLS must be enabled on every table. The following policies are the minimum required. Students must document each policy in their RLS Policy Document submission artifact.

Policy	Rule
profiles — SELECT	Allow all authenticated users to read any profile
profiles — UPDATE	Allow a user to update only their own profile (<code>auth.uid() = id</code>)
posts — SELECT	Allow all authenticated users to read any post
posts — INSERT	Allow authenticated users to insert only their own posts (<code>auth.uid() = user_id</code>)
posts — DELETE	Allow a user to delete only their own posts
likes — SELECT	Allow all authenticated users to read likes
likes — INSERT	Allow authenticated users to insert likes where <code>auth.uid() = user_id</code>
likes — DELETE	Allow a user to delete only their own likes
comments — SELECT	Allow all authenticated users to read comments
comments — INSERT	Allow any authenticated user to insert a comment
comments — DELETE	Allow post owner or comment author to delete a comment
follows — SELECT	Allow all authenticated users to read follow relationships
follows — INSERT	Allow authenticated users to follow where <code>auth.uid() = follower_id</code>
follows — DELETE	Allow a user to unfollow where <code>auth.uid() = follower_id</code>

6. Testing Requirements

Every group must submit passing tests. Tests must be written using `flutter_test`. Mock Supabase dependencies using `mocktail` or `mockito` — do not call the live Supabase endpoint in tests.

Test Type	Target	What to Test
Unit	AuthService	signUp(), signIn(), signOut() return correct results; session is set/cleared properly
Unit	PostRepository	createPost() inserts record; fetchFeed() returns list; deletePost() removes record
Unit	LikeRepository	toggleLike() correctly inserts/deletes; likeCount() returns integer
Unit	CommentRepository	addComment() persists; fetchComments() returns correct post-scoped list
Unit	FollowRepository	follow() inserts; unfollow() deletes; isFollowing() returns bool
Widget	PostCard	Renders image, caption, like count; tapping like button triggers provider method
Widget	CommentList	Renders list of comments; empty state shown when no comments
Widget	LikeButton	Shows correct filled/unfilled icon per state; onTap calls provider
Widget	ProfileAvatar	Renders avatar image from URL; shows placeholder on null URL

Running Tests: flutter test --coverage. All tests must pass with 0 failures before submission. Include the test output (screenshot or text copy) in your GitHub repo under /test_results/.

7. Suggested 14-Day Timeline

This timeline is a recommendation, not a hard per-day gate. However, teams that fall behind the Week 1 Checkpoint (Day 7) typically struggle to complete all six features. Plan accordingly.

Day(s)	Focus & Deliverable
Days 1–2	WEEK 1 KICKOFF — Supabase project setup (auth, storage buckets, initial tables), Flutter project scaffold, Riverpod setup, GoRouter baseline, GitHub repo initialized with README
Days 3–4	Auth flow — registration, login, session persistence, logout; profile table created; unit tests for auth service written
Days 5–6	Profile screen with avatar upload (Supabase Storage); post schema designed; post creation UI with image upload; RLS policies for profiles and posts drafted
Day 7	WEEK 1 CHECKPOINT — Self-review: auth works, profile with avatar loads, posts persist in DB, basic RLS applied. Fix blockers.
Days 8–9	Feed screen with paginated posts; like/react feature with realtime like count updates via Supabase realtime channels
Days 10–11	Comments feature with realtime updates; follow/unfollow system; follower-filtered feed (optional stretch)
Days 12–13	Widget tests for post card, comment list, like button; RLS policy audit and documentation; screen recording walkthrough
Day 14	FINAL SUBMISSION — GitHub push, schema export, dev journal, RLS policy doc, all artifacts compiled and submitted

Team Strategy Tip: Divide the six features across team members by Supabase domain — one member owns Auth + Profile, another owns Posts + Storage, another owns Likes + Realtime, and another owns Comments + Follow + RLS audit. Rotate code reviews daily using GitHub pull requests.

8. Submission Checklist

Every group must submit ALL five artifacts. Missing artifacts result in 0 pts for the Submission Artifacts criterion. Submit via the course LMS with your group number in the submission title.

Artifact 1 — GitHub Repository

- Public repository with the full Flutter project source
- README.md must include: project name, group members, setup instructions, Supabase environment variable setup (.env or --dart-define), and screenshots of the app
- Repository must have at least one commit per team member (shows individual contribution)

Artifact 2 — Supabase Schema Export

- Screenshot of the Supabase Table Editor showing all required tables
- OR a .sql file exported from the SQL Editor showing the full schema with constraints
- Include screenshots of Realtime replication settings (tables enabled for realtime)

Artifact 3 — Screen Recording

- Minimum 3 minutes, maximum 10 minutes
- Must demonstrate: registration, login, avatar upload, post creation with image, liking a post (show realtime update on a second device/simulator), adding a comment, following a user
- Narrated or subtitled walkthroughs are encouraged but not required
- Submit as MP4 or share a Google Drive / YouTube link

Artifact 4 — Developer Reflection Journal

- One journal per team (group submission), minimum 500 words
- Must address: What worked well? What was the hardest Supabase feature to implement and why? What would you do differently? What did each team member contribute?
- Individual reflections are a bonus (not required) but demonstrate deeper engagement

Artifact 5 — RLS Policy Documentation

- A table or document listing every RLS policy implemented
- For each policy: table name, operation (SELECT/INSERT/UPDATE/DELETE), policy SQL expression, and plain-English explanation of what it protects

- Must include a screenshot from the Supabase Dashboard > Authentication > Policies confirming policies are active

9. Grading Rubric (100 Points Total)

Partial credit for Supabase feature criteria (rows 1–6) is at the instructor's discretion based on the screen recording demo and submitted artifacts. A feature that is coded but crashes or does not connect to Supabase is not considered partially implemented – it is not implemented.

Category	Criteria	Full Credit	Partial Credit	Zero
1. Authentication & Session (15 pts)	Email/password signup & login; persistent session on app restart; logout clears session; auth errors handled gracefully	15 pts — All criteria met	8–14 pts — Login/signup works but session persistence or error handling is incomplete (instructor discretion)	0 pts — Auth non-functional
2. Database & CRUD (15 pts)	Posts, comments, likes, and follows persisted in Supabase; all CRUD operations correct; no orphaned data; foreign keys enforced	15 pts — Full schema with proper relations and all CRUD working	8–14 pts — Schema present but some CRUD incomplete or relations missing (instructor discretion)	0 pts — No database integration
3. Realtime Updates (15 pts)	At minimum: live like counts OR live comments update without manual refresh; Supabase realtime channel correctly subscribed and unsubscribed	15 pts — Realtime works on 2+ features	8–14 pts — Realtime works on 1 feature or has minor subscription issues (instructor discretion)	0 pts — No realtime implementation
4. Storage & Uploads (10 pts)	Avatar images uploaded to Supabase Storage; post images uploaded and retrieved from signed/public URLs; no broken image links	10 pts — Both avatar and post image uploads functional	5–9 pts — One upload type works; the other is missing or broken (instructor discretion)	0 pts — No storage integration
5. RLS Policies (15 pts)	SELECT, INSERT, UPDATE, DELETE policies applied to all	15 pts — Correct	8–14 pts — Policies present but	0 pts — No RLS policies applied

Category	Criteria	Full Credit	Partial Credit	Zero
	tables; users cannot read or mutate data they should not own; policies documented in submission	policies on all tables with documentation	incomplete or missing on some tables (instructor discretion)	
6. Riverpod State Mgmt (10 pts)	All reactive state managed via Riverpod providers; no raw setState for Supabase-driven state; providers properly scoped and disposed	10 pts — Riverpod used consistently throughout	5–9 pts — Riverpod used partially; some setState leakage (instructor discretion)	0 pts — Riverpod not used
7. Testing (15 pts)	Unit tests for auth and data layer logic; widget tests for key UI components (post card, comment list, like button); all tests pass	15 pts — Tests cover all 3 areas and pass	8–14 pts — Tests present but incomplete coverage or some failures (instructor discretion)	0 pts — No tests submitted
8. Submission Artifacts (5 pts)	GitHub repo with working README; Supabase schema export; screen recording (3+ min demo); dev journal; RLS policy doc	5 pts — All 5 artifacts complete	2–4 pts — 3–4 artifacts complete	0 pts — Fewer than 3 artifacts

Instructor Discretion Note: For categories 1–6, when a feature is partially implemented, the instructor will evaluate the screen recording and source code to determine the appropriate partial credit score within the stated range. Groups are encouraged to narrate limitations openly in their dev journal — transparency is factored into the evaluation.

10. Academic Integrity & Group Norms

- All code must be original work by the group. AI-assisted code generation is permitted as a learning aid, but students must be able to explain every line of code during a follow-up question from the instructor.
- Copying another group's repository in whole or in part is grounds for a zero on the project for all involved parties.
- GitHub commit history will be reviewed to verify individual contributions. A team member with zero commits will receive a deduction at instructor discretion.

- Supabase credentials (service role key, database password) must NEVER be committed to the repository. Use `--dart-define` or a `.env` file added to `.gitignore`.

11. Reference Resources

Resource	URL
Supabase Flutter Docs	https://supabase.com/docs/reference/dart/introduction
Supabase Auth Docs	https://supabase.com/docs/guides/auth
Supabase Realtime Docs	https://supabase.com/docs/guides/realtime
Supabase Storage Docs	https://supabase.com/docs/guides/storage
RLS Guide	https://supabase.com/docs/guides/auth/row-level-security
Riverpod Docs	https://riverpod.dev/docs/introduction/getting_started
flutter_test Docs	https://docs.flutter.dev/testing
mocktail Package	https://pub.dev/packages/mocktail
image_picker Package	https://pub.dev/packages/image_picker
cached_network_image	https://pub.dev/packages/cached_network_image

— End of Document —